This would certainly require `O(n)` time, where `n` here is the number of nodes in the subtree rooted at the node we remove.

A better solution would be to replace the node `N` that we want to remove with its successor, or its predecessor. In case `N` only has a right child, we can just find its successor using `findMin`, as we described above, in the example in figure 9.16.

When, instead, the node `N` only has a left child, can we replace it with its predecessor? As much as you could be tempted to think so, in this situation another issue comes up.

We mentioned, when describing the insert method, that the way we break ties on insert has an influence on the `remove` method as well.

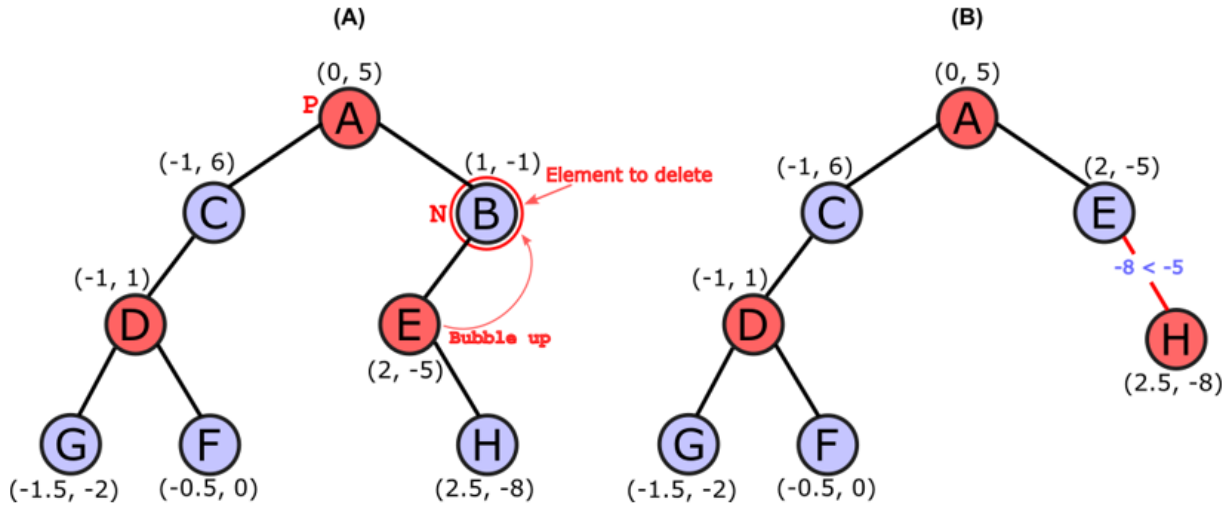


Figure 9.17 An example of a k-d tree deletion for which replacing the deleted node with its only child wouldn’t work.
(A) The initial tree, from which we remove node `B`.